# CSC3066 Deep Learning

## Large Language Models

Lu Bai (l.bai@qub.ac.uk)

# Content of Today Lecture

- Introduction to Large Language Models (LLMs)
- Transformer model
  - Attention mechanism
  - Encoder
  - Decoder
- Overview of transformer based LLMs
- BERT model

# Introduction

# Large Language Models

- LLM is a trained deep-learning model that understands and generate text in a human-like fashion.
- LLMs are based on a transformer architecture and are trained using massive datasets comprising billions of words or more.
- LLMs generate contextualized embeddings that capture the meaning and context of words based on their surroundings.

# Large Language Models

- Unlike traditional word embeddings (e.g., Word2Vec, GloVe), which assign fixed representations to words, contextual embeddings adapt based on context, leading to more accurate and nuanced understanding of language.
- After pre-training, LLMs can be fine-tuned for specific downstream tasks, including text classification, named entity recognition, question answering, and language generation.
- LLMs have achieved state-of-the-art performance across a wide range of NLP benchmarks and tasks.

# Transformer Model

# Transformers

- The Transformer model was proposed by google in 2017

**Attention Is All You Need**


**Ashish Vaswani*** Google Brain avaswani@google.com

**Noam Shazeer*** Google Brain noam@google.com

**Niki Parmar*** Google Research nikip@google.com

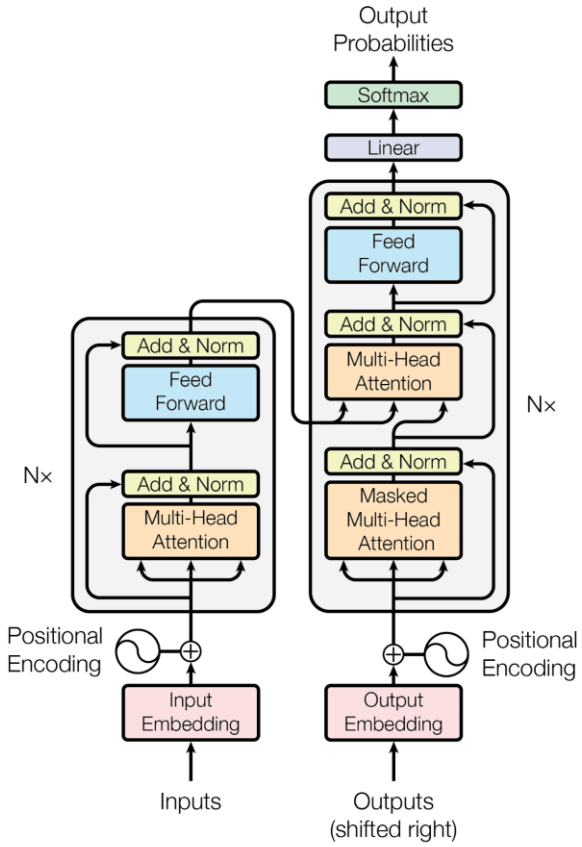
**Jakob Uszkoreit*** Google Research usz@google.com

**Llion Jones*** Google Research llion@google.com

**Aidan N. Gomez*** † University of Toronto aidan@cs.toronto.edu

**Łukasz Kaiser*** Google Brain lukaszkaiser@google.com

**Illia Polosukhin*** ‡ illia.polosukhin@gmail.com

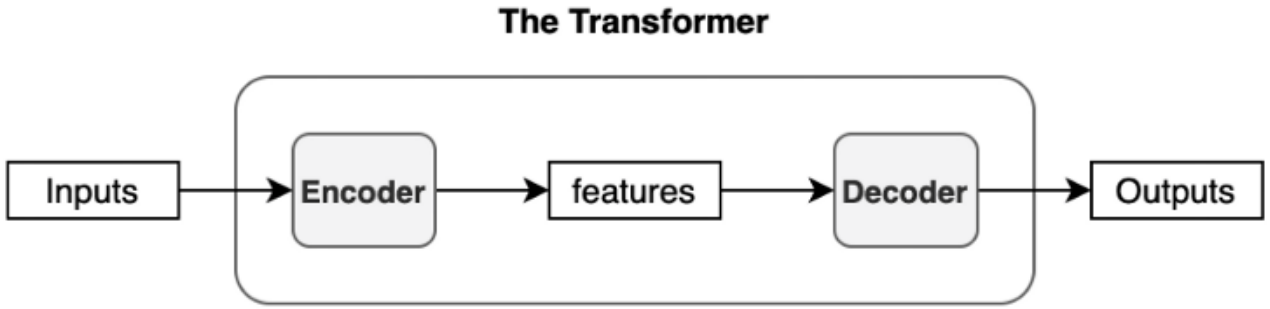

https://arxiv.org/abs/1706.03762

# Transformers

- Transformer is a sequence-to-sequence model (Seq2Seq), which is a neural network that transforms a given sequence of elements, such as the sequence of words in a sentence, into another sequence.

# Transformers: Architecture

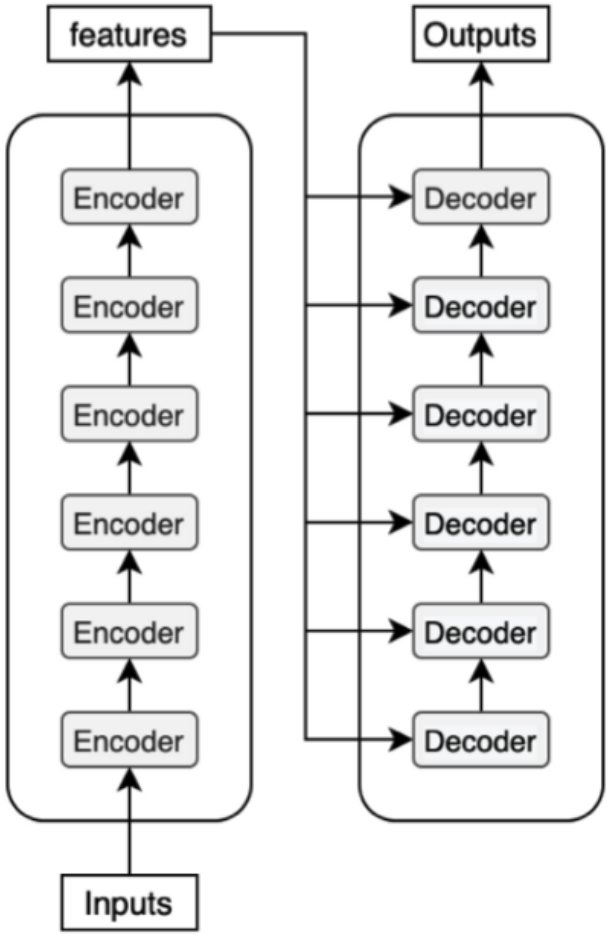
- Transformer is composed of two key components: Encoder and Decoder




- It applies self-attention mechanism, which allows the model to weigh the importance of different words in a sequence by considering the relationships between all pairs of words in the sequence.

# Transformers: Encoder and Decoder

- The encoder is responsible for processing the input sequence and generating a set of context-aware representations for each token in the sequence.
- The decoder takes the encoder's output and generates the target sequence, token by token, while attending to the encoder's output and previously generated tokens.



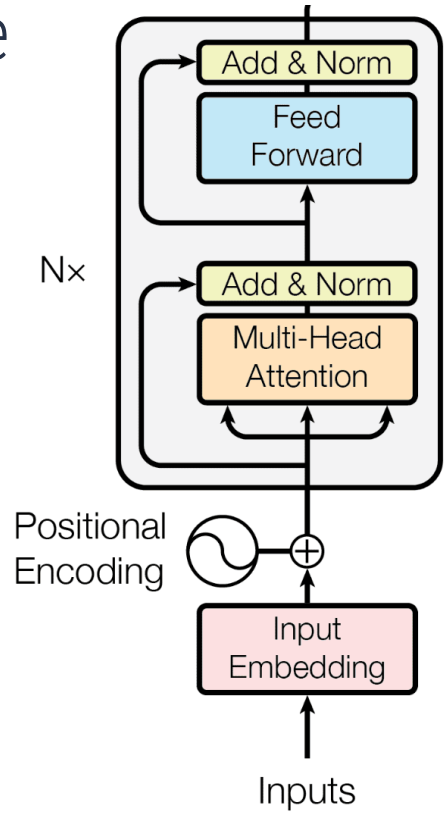
# Transformers: Self-Attention Mechanism

- In ANNs, the concept of attention mimics the human cognitive process of selectively focusing on certain parts of input data while ignoring others.
- The attention mechanism works by assigning importance weights to different elements of the input data, enabling the model to focus on the most relevant parts during processing.
- In the context of the Transformer model, self-attention allows the model to weigh the importance of different words in a sequence by considering the relationships between all pairs of words in the sequence.

# Encoder

# Encoder

- The encoder is responsible for processing the input sequence and generating a set of context-aware representations for each token in the sequence




https://arxiv.org/abs/1706.03762

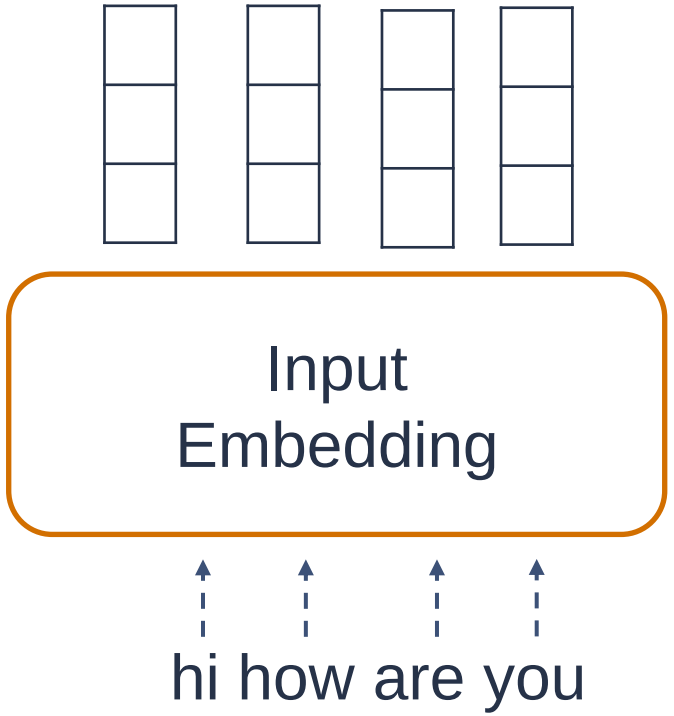
# Encoder: Positional Encoding

# Encoder: Encoding Input Data

- Input Embeddings: the input to the Transformer model consists of tokenized sequences of words, where each word is represented as an embedding vector.

Hi=

| |
|---|
| 0.41 |
| 0.98 |
| 0.12 |



# Encoder: Positional Encoding

- Positional Encoding (PE) injects positional information into embeddings, allowing the model to capture sequential dependencies.
- Positional Encoding is done using sin and cosine functions:

$$PE(k, 2i) = sin\left(\frac{k}{n^{2i/d}}\right)$$

$$PE(k, 2i+1) = cos\left(\frac{k}{n^{2i/d}}\right)$$

As the results we obtained Positional Encoding Matrix, which is added to the input embeddings

# Encoder: Positional Encoding

$$PE(k, 2i) = sin\left(\frac{k}{n^{2i/d}}\right)$$

$$PE(k, 2i+1) = cos\left(\frac{k}{n^{2i/d}}\right)$$

$k$: position of an object in the input sequence, $0 \leq k \leq L - 1$

$L$: length of the input sequence

$d$: dimension of the output embedding space

$PE(k, j)$: position function for mapping a position $k$ in the input space to index $(k, j)$ of the positional matrix

$n$: user defined scalar (set to 10,000 by the authors of Attention Is All You Need)

$i$: Used for mapping to column indices $0 \leq i \leq {}^{d}/_{2}$, with a single value of $i$ maps to both sine and cosine functions

# Encoder: Positional Encoding

$$PE(k, 2i) = sin\left(\frac{k}{n^{2i/d}}\right)$$

$$PE(k, 2i+1) = cos\left(\frac{k}{n^{2i/d}}\right)$$

## Positional Encoding Matrix with $d = 4$ and $n = 100$

| $Sequence$ | | $k$ | | $i = 0$ | $i = 0$ | $i = 1$ | $i = 1$ |
|---|---|---|---|---|---|---|---|
| Hi | → | 0 | → | $P_{00} = \sin(0) = 0$ | $P_{01} = \cos(0) = 1$ | $P_{02} = \sin(0) = 0$ | $P_{03} = \cos(0) = 1$ |
| How | → | 1 | → | $P_{10} = \sin(1/1) = 0.84$ | $P_{11} = \cos(1/1) = 0.54$ | $P_{12} = \sin(1/10) = 0.10$ | $P_{13} = \cos(1/10) = 1$ |
| Are | → | 2 | → | $P_{20} = \sin(2/1) = 0.91$ | $P_{21} = \cos(2/1) = -0.42$ | $P_{22} = \sin(2/10) = 0.20$ | $P_{21} = \cos(2/10) = 0.98$ |
| You | → | 3 | → | $P_{30} = \sin(3/1) = 0.14$ | $P_{31} = \cos(3/1) = -0.99$ | $P_{32} = \sin(3/10) = 0.30$ | $P_{31} = \cos(3/10) = 0.96$ |

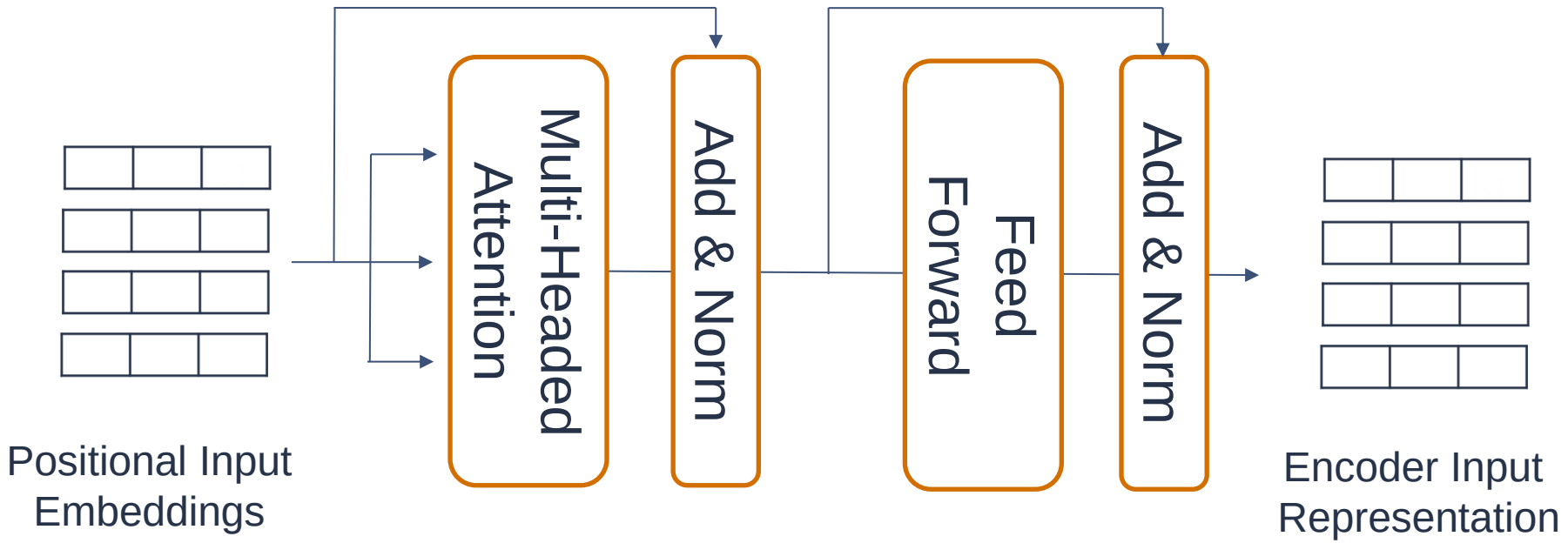
# Encoder: Positional Encoding

| | | | | |
|---|---|---|---|---|
| Input sequence → | Hi | How | Are | You |
| Word embedding → | $v_o$ =embedding vector (Hi) | $v_1$ =embedding vector (How) | $v_2$ =embedding vector (Are) | $v_3$ =embedding vector (You) |
| | | + | | |
| Positional Encoding Matrix → | $P_o$ =positional vector (Hi) | $P_1$ =positional vector (How) | $P_2$ =positional vector (Are) | $P_3$ =positional vector (You) |
| | | = | | |
| Output of positional encoding layer → | $y_o$ =positional encoding (Hi) | $y_1$ =positional encoding (How) | $y_2$ =positional encoding (Are) | $y_3$ =positional encoding (You) |

# Encoder: Multi-headed Self Attention Mechanism

# Encoder: Multi-head Self Attention Mechanism

- Encoder Layer contains two main sub-layers:
  - Multi-headed Self Attention Mechanism
  - Feed Forward Network

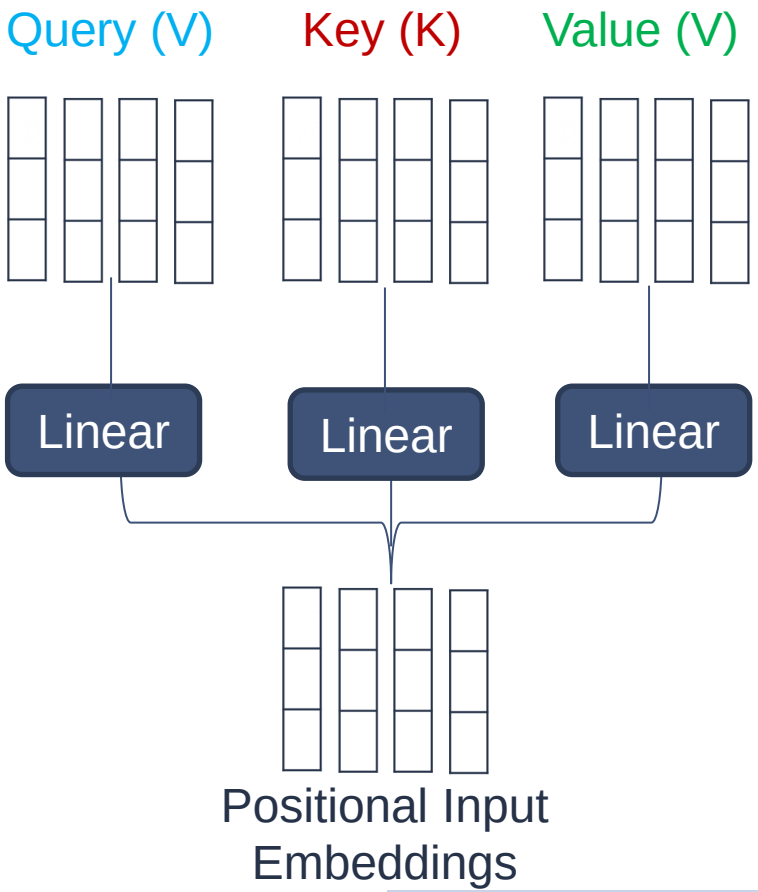


# Encoder: Self Attention Mechanism

- The self-attention layer helps the encoder look at other words in the input sentence as it encodes a specific word
- Given the sentence: *"I arrived at the bank after crossing the river"*, to determine that the word "bank" refers to the shore of a river and not a financial institution, the Transformer can learn to immediately pay attention to the word "river" and make this decision in just one step.

# Encoder: Self Attention Mechanism

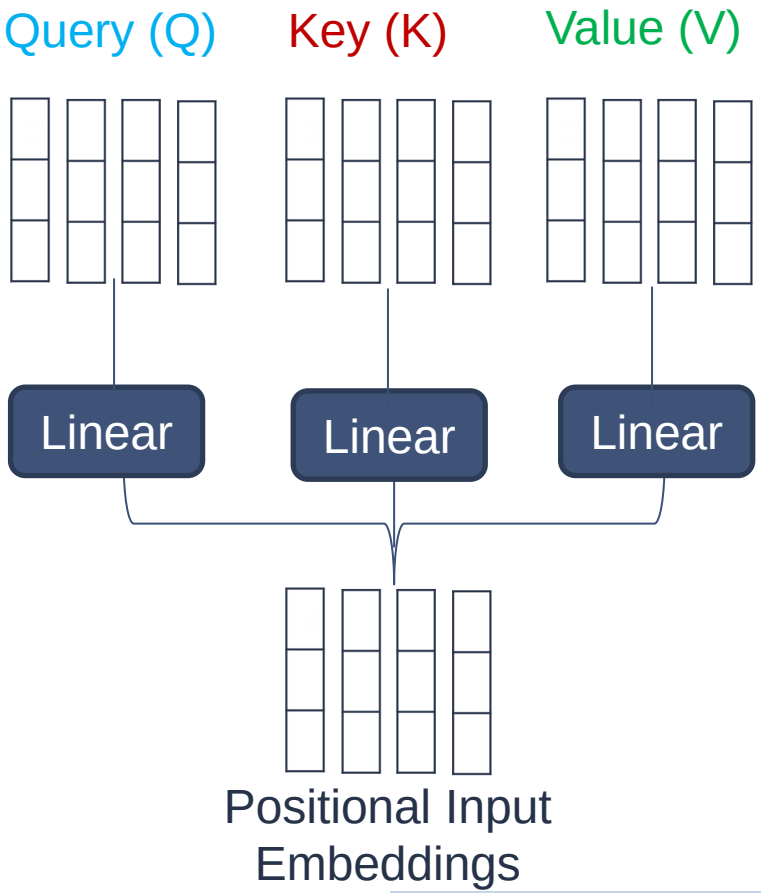
- To compute self-attention, the input embeddings are fed into three distinct fully connected layers to create: query vectors, key vectors, and value vectors.
- These transformations are learned during the training process and are applied linearly to the input embeddings.



# Encoder: Self Attention Mechanism

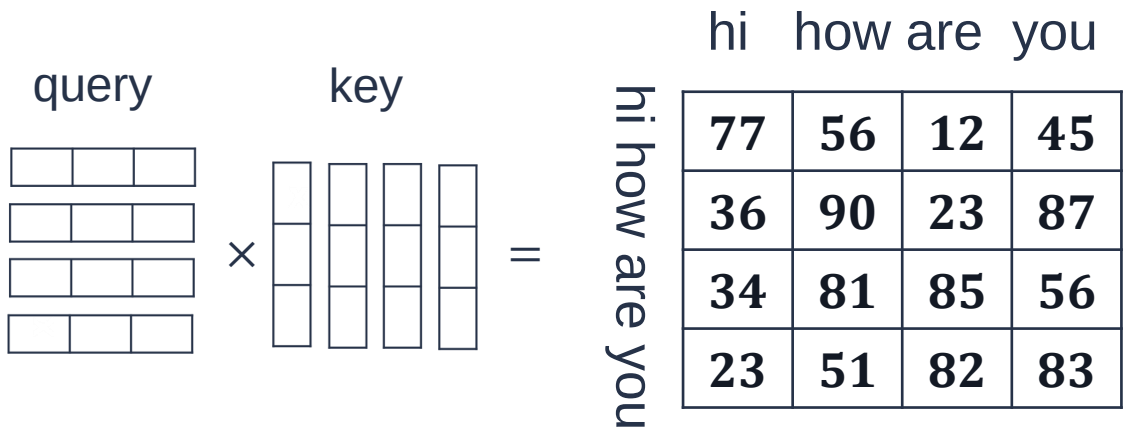
- Query vectors: representation of the current word being process
- Key vectors: representation of a word with respect to its relevance (importance) to other words
- Value vectors: holds information about the importance of each word (its value)
- Attention: (output vector) weighted sum of the values

$$Attention = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$



# Encoder: Self Attention Mechanism

- Computing Attention Scores: the queries and keys undergo a dot product matrix multiplication to produce a score matrix.
- The score matrix determines how much focus should a word be put on other words.



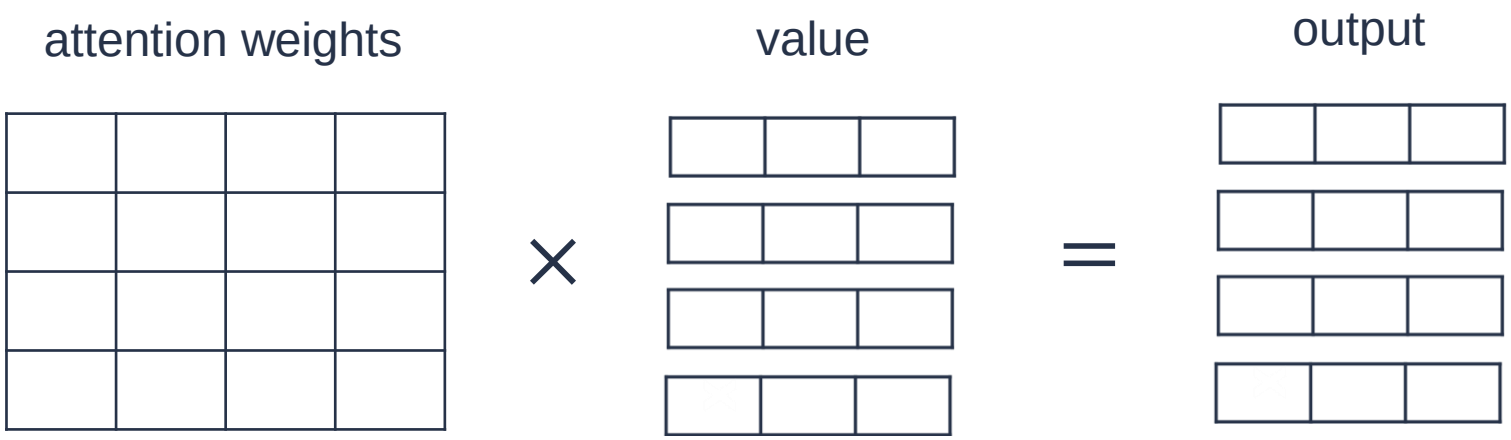
# Encoder: Self Attention Mechanism

- Scaling Down the Attention Scores: the scores are scaled by getting divided by the square root of the the dimension of query and key
- Softmax of the Scaled Scores provides attention weights

$$Softmax\left(\frac{Attention\ Scores}{\sqrt{d}}\right) = \begin{bmatrix} 0.7 & 0.1 & 0.1 & 0.1 \\ 0.1 & 0.6 & 0.2 & 0.1 \\ 0.1 & 0.3 & 0.5 & 0.1 \\ 0.1 & 0.3 & 0.3 & 0.3 \end{bmatrix}$$

# Encoder: Self Attention Mechanism

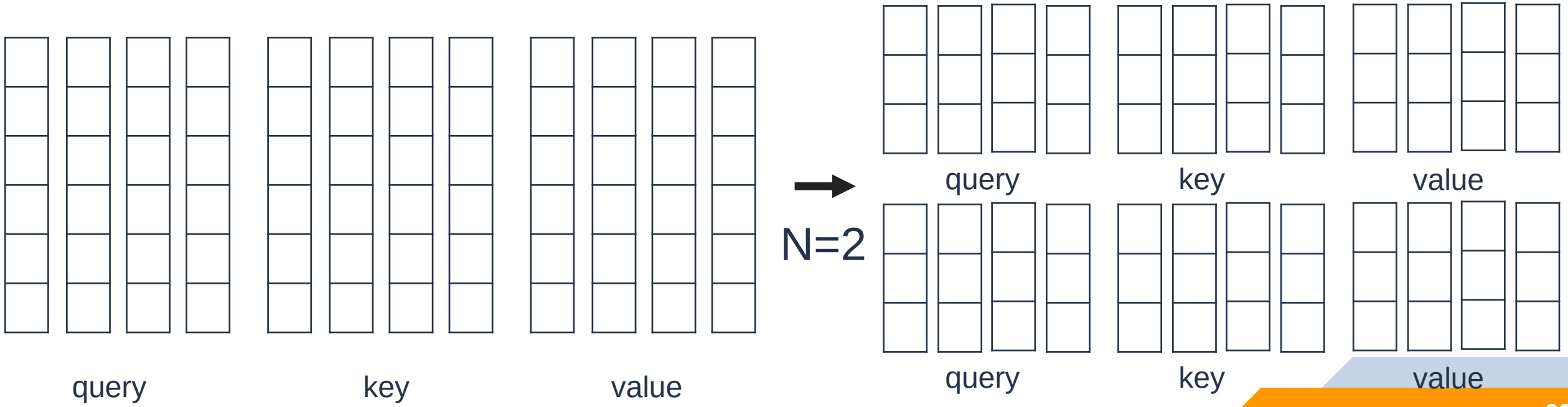
## Multiplying Softmax Output with Value vector




The weighted sum of the value vectors, where the weights are determined by the attention scores, produces the output of each attention head.

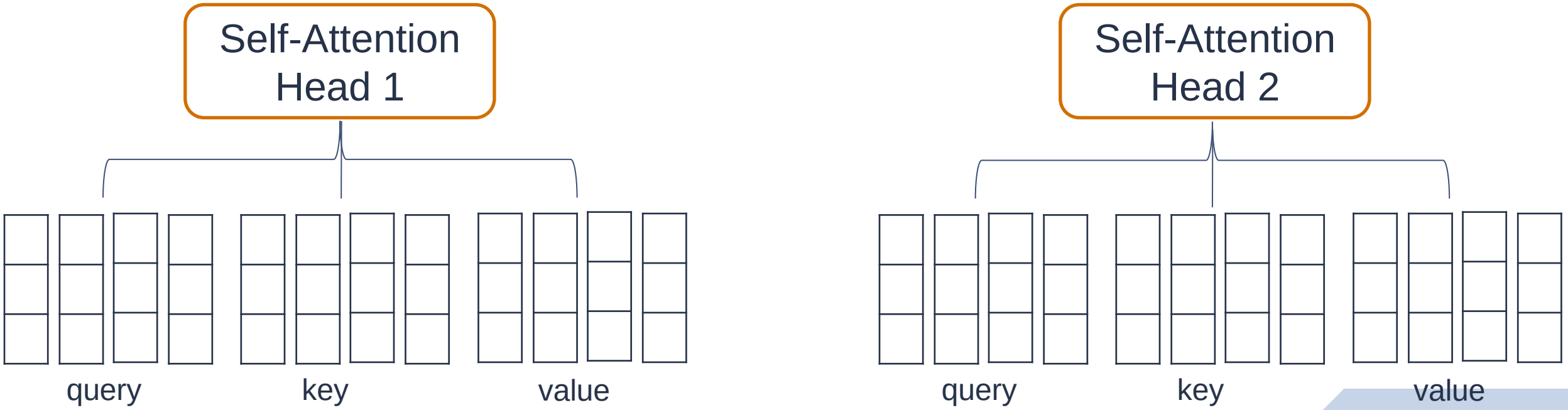
# Transformers: Multi-headed Self Attention Mechanism

Multi-headed Attention is computed by splitting the query, key and value vectors into N vectors before applying self-attention.



# Transformers: Multi-headed Self Attention Mechanism

The split vectors go through the self-attention process (head) individually.

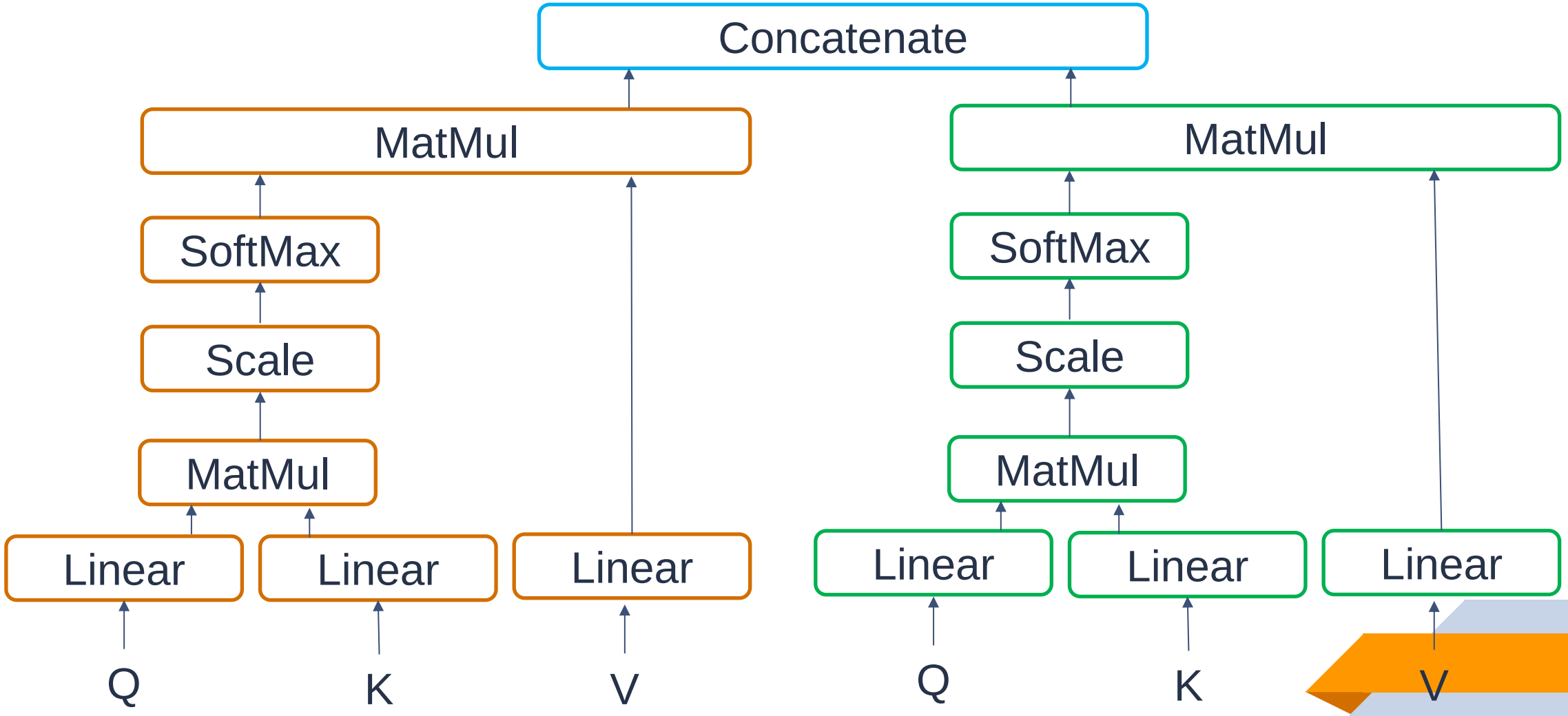



N=2

# Transformers: Multi-headed Self Attention Mechanism

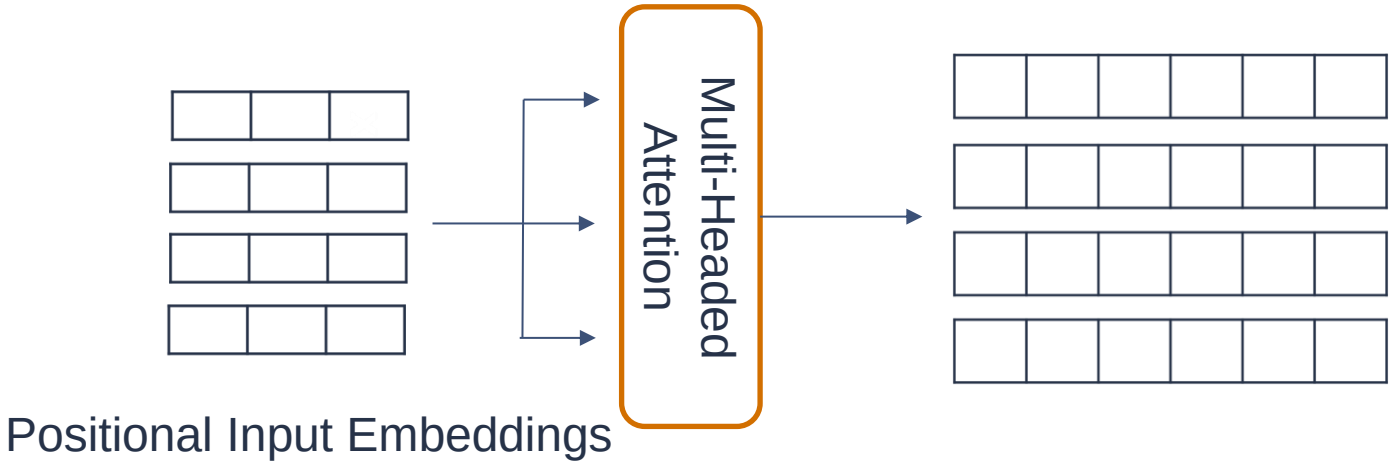
- Each attention head learns its own set of query, key, and value vectors through a linear transformation of the input embeddings.
- Each attention head learns to attend to different parts of the input sequence, capturing different relationships and patterns.
- Each attention head produces an output vector that get concatenated into a single vector.

# Transformers: Multi-headed Self Attention Mechanism



# Transformers: Multi-headed Self Attention Mechanism

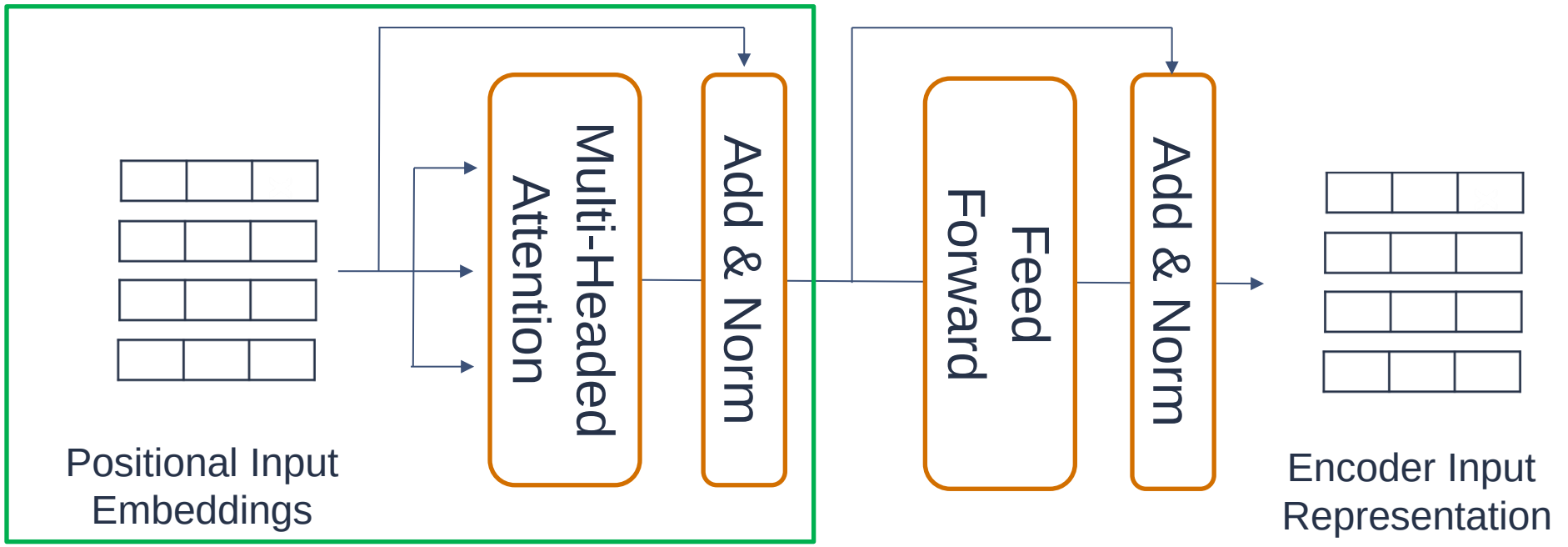
- The output of multi-headed attention is a set of vectors, each of which represents a weighted combination of the input values, where the weights are determined by the attention scores computed from the queries and keys for each head. These output vectors capture different aspects of the input data and are then used as input to subsequent layers in the transformer model.



# Encoder: Residual Connections, Layer Normalization and Feed Forward Network

# Encoder: Residual Connections, Layer Normalization and Feed Forward Network

- The multi-headed attention output vector is added to the original positional input embedding. This is called a residual (skip) connection. ???
- The output vector is passed through a layer normalization (activation normalization for more stable training process).



# Encoder: Residual Connections, Layer Normalization and Feed Forward Network

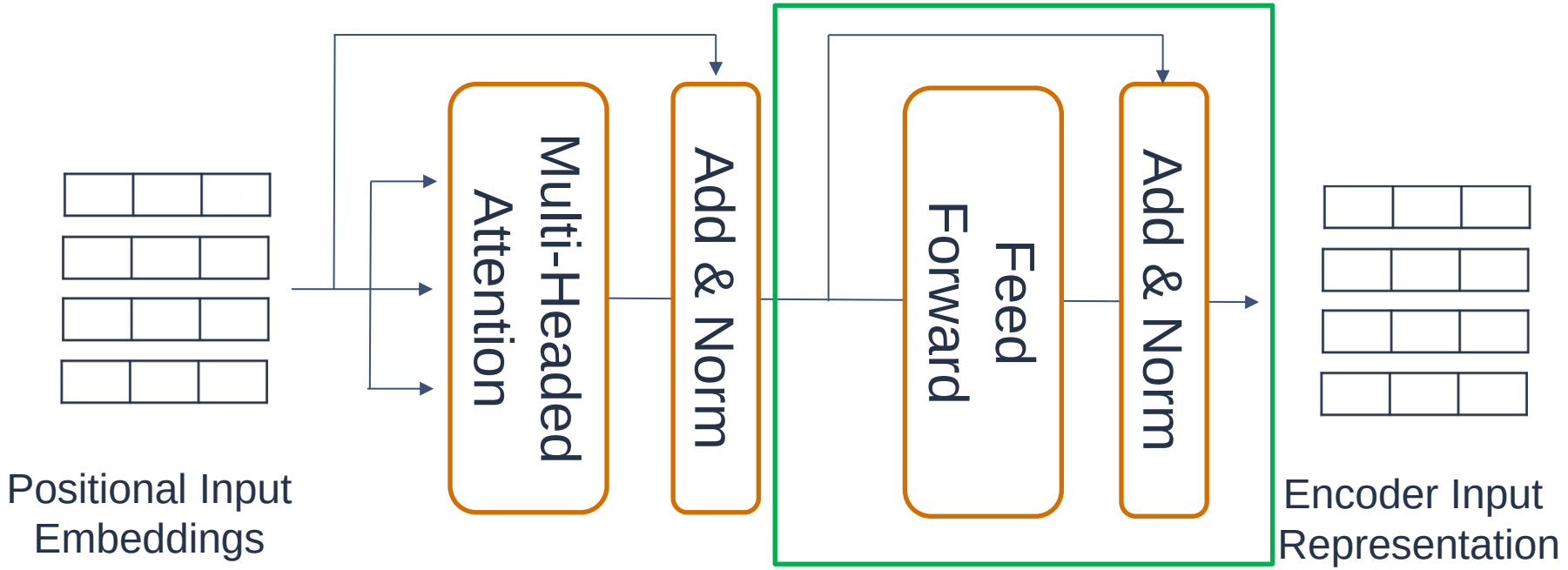
- Skip connections, serve several important purposes in transformers and other deep learning architectures:
  - Addressing Vanishing Gradient Problem: Deep neural networks often encounter the vanishing gradient problem, where gradients become very small as they propagate through many layers during training. This can hinder the training process, making it difficult for the network to learn effectively. Residual connections help mitigate this issue by providing shortcuts for gradient flow. By directly connecting the input of a layer to its output, residual connections ensure that gradients can flow more easily through the network, facilitating training of deep models.

# Encoder: Residual Connections, Layer Normalization and Feed Forward Network

- Skip connections, serve several important purposes in transformers and other deep learning architectures:
  - Learning Identity Mapping: In addition to mitigating the vanishing gradient problem, residual connections also enable the network to learn identity mappings. If a layer learns to do nothing useful, the residual connection allows the information to bypass the layer and propagate directly to the subsequent layers. This ensures that the model can learn to add new information to the input rather than being forced to learn from scratch at each layer.

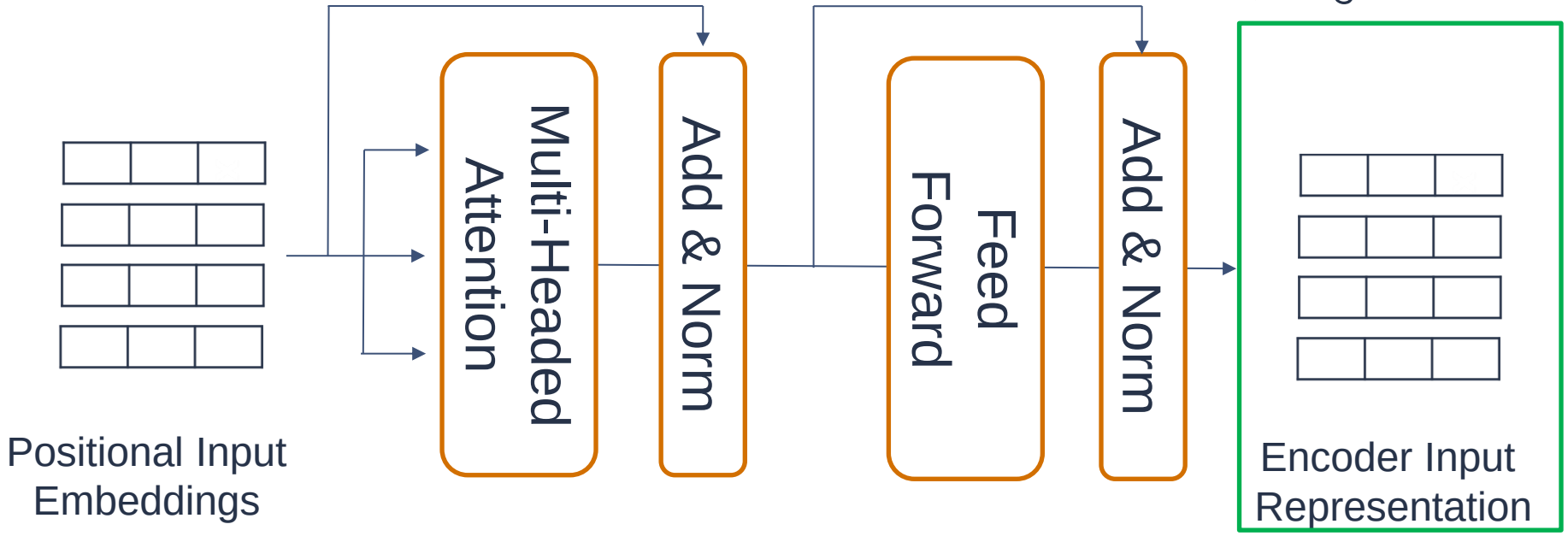
# Transformers: Residual Connections, Layer Normalization and Feed Forward Network

- The normalized residual output gets projected through a feed-forward network for further processing (a couple of linear layers with a ReLU activation in between).
- Layer normalization and residual connections are applied again.



# Transformers: Residual Connections, Layer Normalization and Feed Forward Network

- The final output of the encoder is a sequence of contextualized representations for each token in the input sequence.
- These representations capture both the semantic meaning and the positional information of the tokens, which can then be used for downstream tasks like classification, translation, or generation.

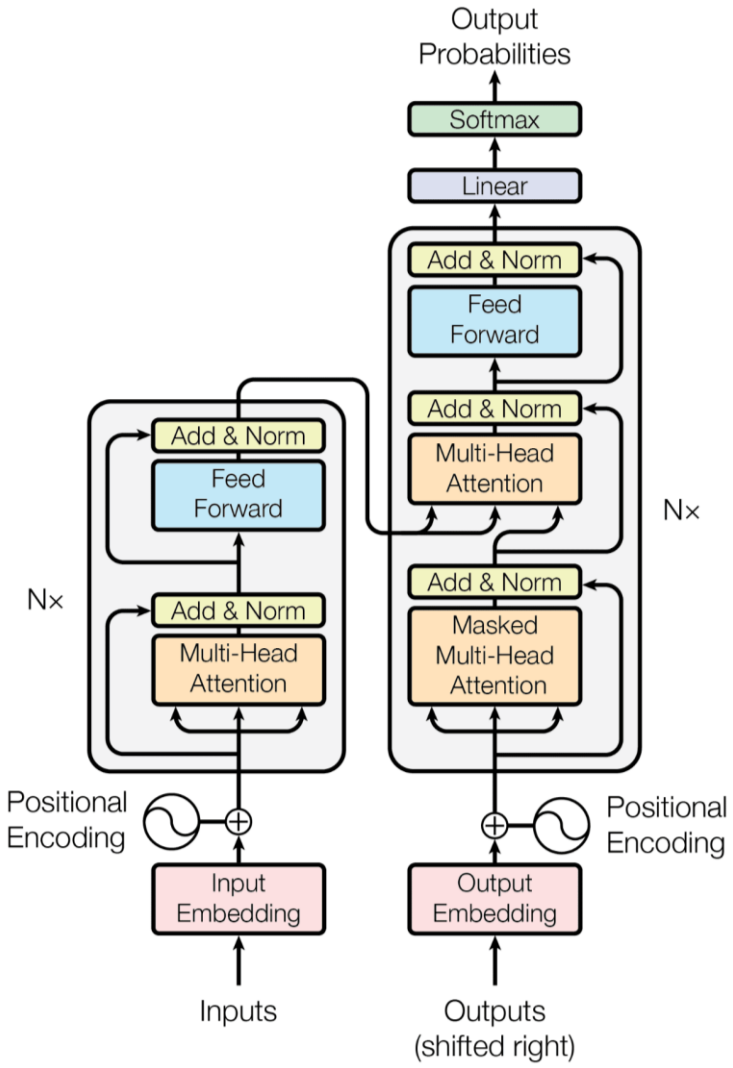


# Decoder

# Transformers: Decoder

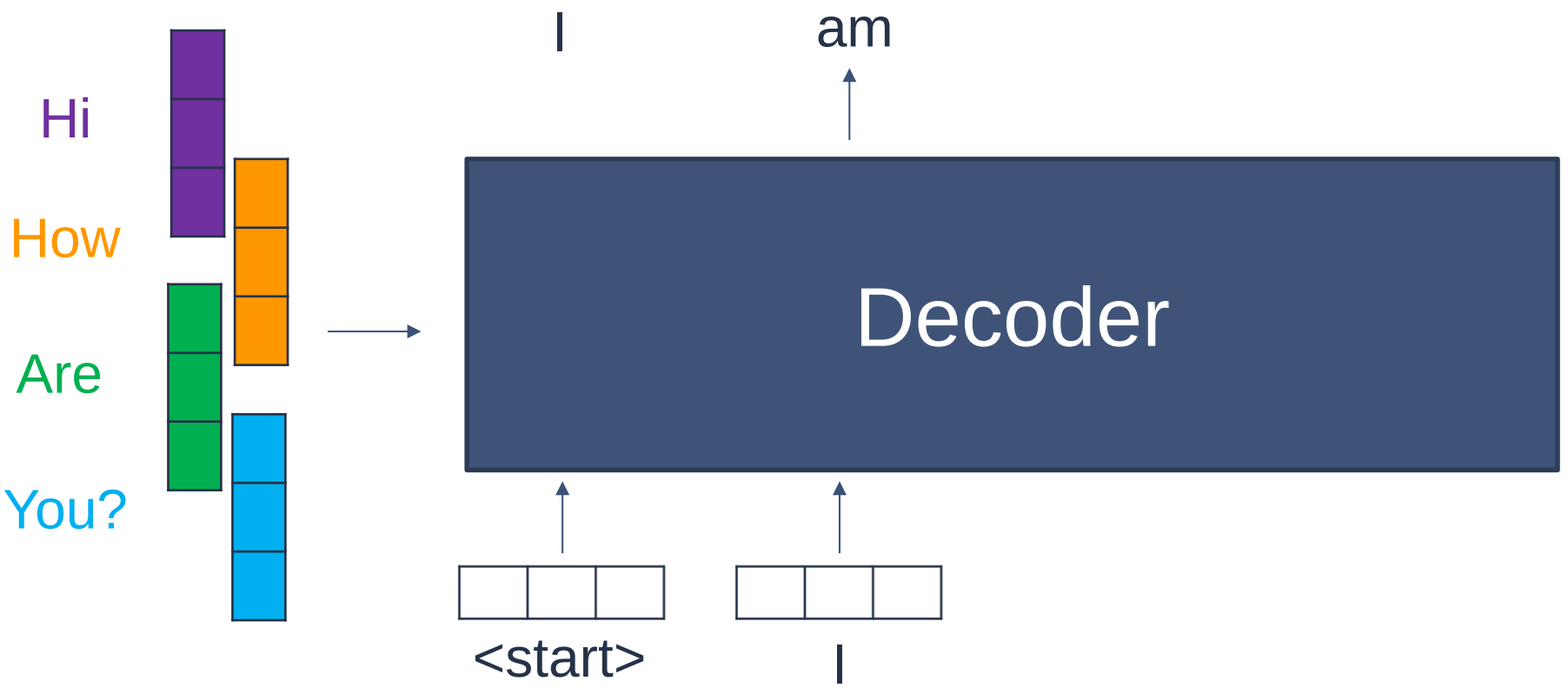
- The decoder part of the Transformer model is responsible for generating output text sequences based on the representations learned by the encoder.
- It has similar structure to the encoder including two multi-headed attention layers, a feed-forward layer, residual connections and layer normalization.
- It takes in a list of previous outputs as inputs, as well as the encoder outputs that contain the attention information from the input.

# Transformers: Decoder



# Transformers: Decoder

- Decoder takes in a list of previous outputs as inputs, as well as the encoder outputs that contain the attention information from the input.



# Decoder: Positional Encoding

# Decoder: Positional Encodings

- Target Sequence Embeddings:
  - The decoder takes as input the embeddings of the tokens in the target sequence. These embeddings are similar to the word embeddings used in the encoder but represent the tokens in the target sequence rather than the input sequence.
- Positional Encodings:
  - Similar to the encoder, the decoder adds positional encodings to the target sequence embeddings to provide information about the position of tokens in the decoder's input sequence.

# Decoder: First Multi-Headed Attention

# Decoder: First Multi-Headed Attention

- First (Masked) Multi-Headed Attention: opposite to Encoder, in Decoder enables the decoder to capture dependencies within the target sequence, ensuring that each token's representation incorporates information from preceding tokens.
- Unlike the encoder's Multi-Head Self-Attention Mechanism, which attends to all tokens in the input sequence, the decoder's masked self-attention mechanism only allows each token to attend to its preceding tokens.
- This prevents the decoder from having access to future information during training.

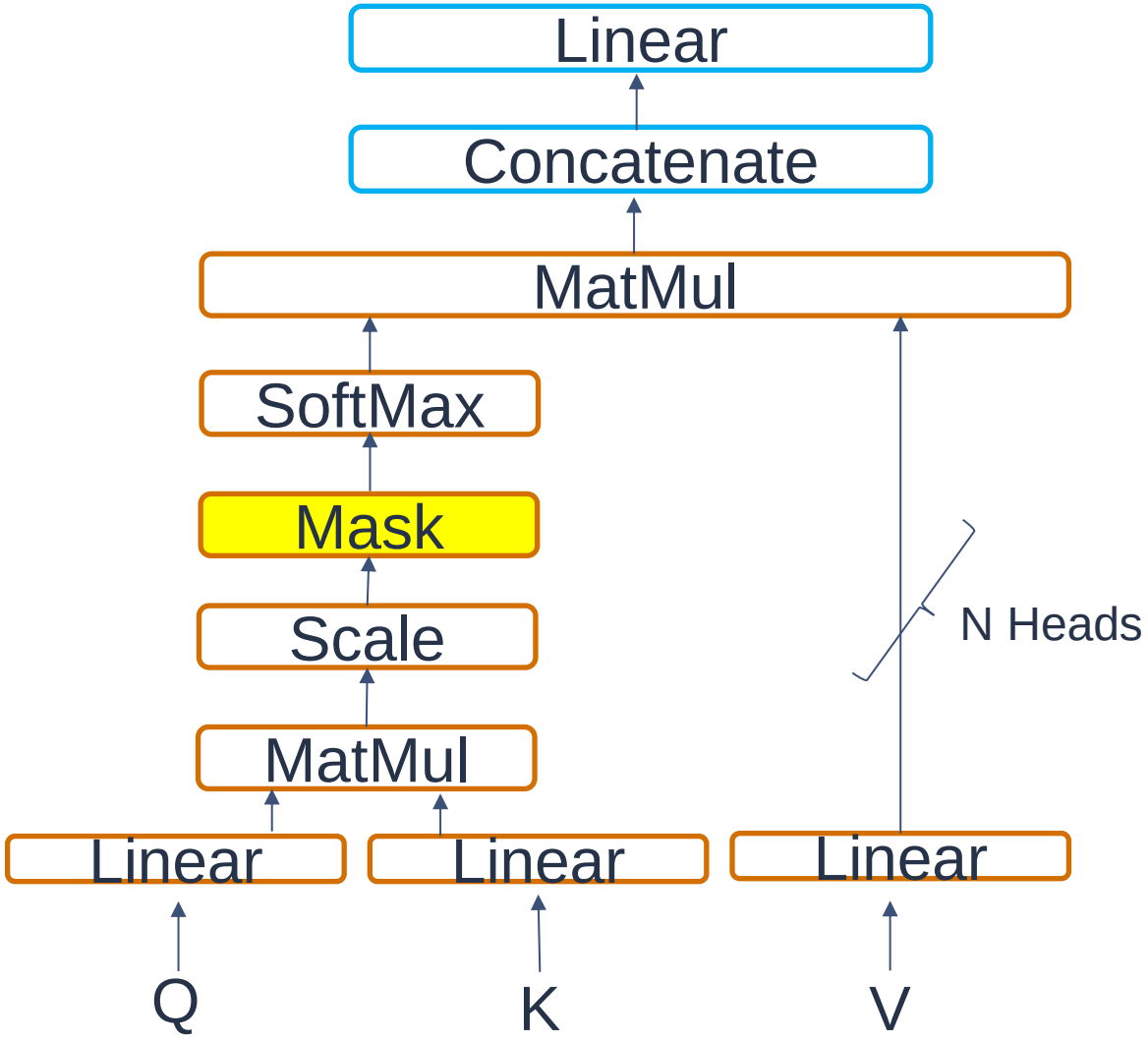
# Decoder: First Multi-Headed Attention

- In the decoder's masked self-attention mechanism, the attention scores for future tokens are set to a very large negative value (usually -infinity) before applying the softmax function.
- This ensures that the softmax operation effectively masks out the future tokens, assigning zero attention weight to them.

$$Softmax\left(\frac{Attention\ Scores}{\sqrt{d}} + \begin{bmatrix} 0 & -\inf & -\inf & -\inf \\ 0 & 0 & -\inf & -\inf \\ 0 & 0 & 0 & -\inf \\ 0 & 0 & 0 & 0 \end{bmatrix}\right) = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0.3} & \mathbf{0.9} & \mathbf{0} & \mathbf{0} \\ \mathbf{0.1} & \mathbf{0.5} & \mathbf{0.4} & \mathbf{0} \\ \mathbf{0.1} & \mathbf{0.3} & \mathbf{0.3} & \mathbf{0.3} \end{bmatrix}$$

Look-Ahead Mask

# Decoder: First Multi-Headed Attention



# Decoder: Second Multi-Headed Attention

# Decoder: Second Multi-Headed Attention

- Second Multi-Headed Attention: allows the decoder to attend to different parts of the encoder's output while generating each token in the target sequence.
- For this layer, the encoder’s outputs are the keys and values, and the first multi-headed attention layer outputs are the queries.
- The decoder generates queries based on its current state, aiming to attend to relevant information in the encoder's output while generating each token of the target sequence.
- The output of the second multi-headed attention goes through a feedforward layer for further processing.

# Decoder: Residual Connections and Layer Normalization

# Transformers: Residual Connections and Layer Normalization

- Similarly to the encoder, each sub-layer in the decoder is followed by a residual connection, which adds the input to the output of the sub-layer.
- Layer normalization is applied after each sub-layer to stabilize training and improve the flow of gradients through the network.

# Decoder: Linear Classifier and Final Softmax

# Decoder: Linear Classifier and Final Softmax

- The output of the final feedforward layer goes through a final linear layer, that acts as a classifier.
- The classifier is as big as the number of classes you have. For example, if you have 10,000 classes for 10,000 words, the output of that classier will be of size 10,000.

# Decoder: Linear Classifier and Final Softmax

- The output of the classifier then gets fed into a softmax layer, which will produce probability scores between 0 and 1.
- We take the index of the highest probability score, and that equals our predicted word.
- The decoder then takes the output, adds it to the list of decoder inputs, and continues decoding again until a token is predicted.

# Transformer Based LLM

# Examples of Transformer-based LLM

- Transformer-based large language models represent a significant advancement in NLP and have revolutionized various NLP tasks.
- Various transformer based large language models have been proposed.
- Different LLM may leverage different components of the transformer's architecture, depending on the application.

# Examples of Transformer-based LLM

## BERT (Bidirectional Encoder Representations from Transformers)

- **Usage**: Encoder.
- **Description**: BERT is pre-trained on large corpora using masked language modelling and next sentence prediction tasks. It captures bidirectional context by considering both left and right context during pre-training.
- **Variants**: BERT-base, BERT-large, and domain-specific versions like BioBERT and ClinicalBERT.

# Examples of Transformer-based LLM

## GPT (Generative Pre-trained Transformer):

- **Usage**: Decoder.
- **Description**: The GPT series (including GPT, GPT-2, GPT-3 and GPT-4) are autoregressive language models trained using unsupervised learning objectives. They generate text autoregressively, predicting the next token given the previous context.

# Examples of Transformer-based LLM

## T5 (Text-to-Text Transfer Transformer):

- **Usage**: Both encoder and decoder
- **Description**: T5 follows a unified text-to-text framework where all NLP tasks are framed as text-to-text transformation tasks. It is trained on a diverse range of tasks and achieves state-of-the-art results across various benchmarks

# BERT Model

# BERT Model

- BERT is a pre-trained language model developed by Google in 2018.
- It is powered by the Transformer architecture, which incorporates the self-attention mechanism allowing BERT to generate contextualised word embeddings.
- BERT's goal is to generate a language representation model hence it only needs the encoder part.

# BERT Model

- BERT's contextualized word representations have found applications across a wide range of NLP tasks, including:
  - Text Classification
  - Named Entity Recognition
  - Question Answering
  - Sentiment Analysis

# BERT Model
# Data Pre-processing

# BERT: Data Pre-Processing

- The input to the encoder for BERT is a sequence of tokens, which are first converted into vectors
- Data pre-processing in BERT involves several important steps to prepare the text data for training or fine-tuning the model.
  - Tokenisation
  - Special Tokens
  - Token Embeddings
  - Segment Embeddings
  - Positional Embeddings

# BERT: Data Pre-Processing

## Tokenization:

- BERT employs the WordPiece tokenizer to break down text into word and some words into smaller subword units based on the pre-defined vocabulary.
- Each word is assigned a token ID, and rare or out-of-vocabulary words are broken down into subword units that exist in the vocabulary (e.g. ‘sleeping’ -> ‘sleep’ and ‘##ing’).

"BERT is a powerful NLP model.”

["BER", "##T", "is", "a", "powerful", "N", "##LP", "model", "."]

# BERT: Data Pre-Processing

## Special Tokens:

- BERT adds special tokens to the beginning and end of each input sequence.
- The [CLS] token (classification token) is added at the beginning of the sequence and represents the entire input sequence for classification tasks.
- The [SEP] token (separator token) is used to separate pairs of sentences in tasks such as sentence pair classification and question answering.

"BERT is a powerful NLP model." and "It is widely used in natural language processing."

[CLS] "BERT" "is" "a" "powerful" "N" "##LP" "model" "." [SEP] "It" "is" "widely" "used" "in" "natural" "language" "processing" "." [SEP]

# BERT: Data Pre-Processing

## Token Embeddings:

- After tokenization, each token (which could be a word or subword unit) is mapped to its corresponding token ID in the BERT vocabulary.
- Once the tokens are mapped to token IDs, BERT looks up the corresponding token embeddings from an embedding table.
- These embeddings are initialized randomly at the beginning of training and are fine-tuned during the training process.

# BERT: Data Pre-Processing

## Segment Embeddings:

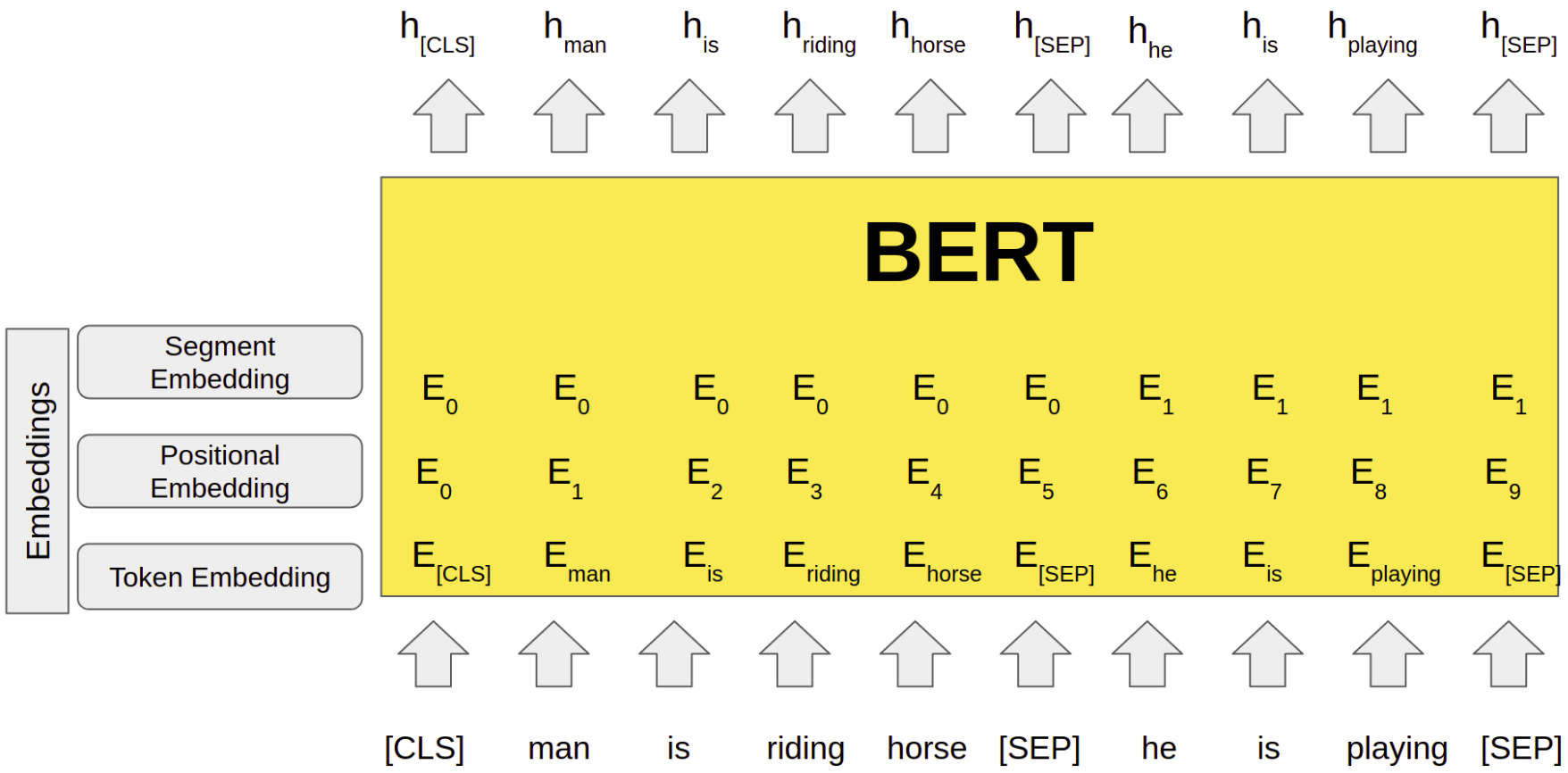
- BERT can accept input sequences consisting of pairs of sentences (e.g., question-answer pairs)
- Segment embeddings are used in BERT to distinguish between different segments or sentences within the input sequence.
- To differentiate between the segments, BERT assigns a segment embedding of 0 or 1 to each token, indicating whether the token belongs to the first segment (segment A) or the second segment (segment B).

# BERT: Data Pre-Processing

## Positional embeddings :

- Positional embeddings encode the position or order of tokens within the input sequence.
- They enable BERT to understand the sequential order of tokens in the input sequence and capture positional dependencies.
- Positional embeddings are typically represented as sine and cosine functions of different frequencies to encode positional information efficiently.

# BERT: Data Pre-Processing




https://www.analyticsvidhya.com/blog/2021/05/all-you-need-to-know-about-bert/

# BERT Model
# Pre-Training Objectives

# BERT: Pre-Training Objectives

BERT uses two unsupervised pre-training objectives:

- Masked Language Modelling: A certain percentage of input tokens are randomly masked, and the model is trained to predict the masked tokens based on the surrounding context.
- Next Sentence Prediction: BERT predicts whether a pair of sentences appear consecutively in the original text.

# BERT: Pre-Training Objectives

## Masked Language Modelling (MLM):

- During pre-training, a certain percentage (typically 15%) of input tokens are randomly selected to be masked.
- BERT replaces these selected tokens with a special [MASK] token.

[CLS] the man went to a store [SEP] he bought a gallon of milk

[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk

# BERT: Pre-Training Objectives

## Masked Language Modelling (MLM):

- The objective of Masked Language Modelling is to predict the original identity of the masked tokens based on the surrounding context.
- For each masked token in the input sequence, BERT predicts the probability distribution over the entire vocabulary for the correct token.
- BERT computes the loss between the predicted probabilities and the actual tokens for all masked positions in the input sequence.

# BERT: Next Sentence Prediction

Next Sentence Prediction: is designed to help BERT understand the relationships between pairs of sentences and capture discourse-level information

- During pre-training, BERT is fed with pairs of sentences as input. Each input sequence consists of two segments:
- **Segment A:** The first sentence in the pair.
- **Segment B:** The second sentence in the pair (or a randomly selected sentence that follows the first sentence in the original text).

# BERT: Next Sentence Prediction

## Next Sentence Prediction:

- For each input pair, BERT is provided with a label indicating whether Segment B follows Segment A in the original text. The label is:
- 'IsNext': Segment B immediately follows Segment A in the original text.
- 'NotNext': Segment B does not immediately follow Segment A

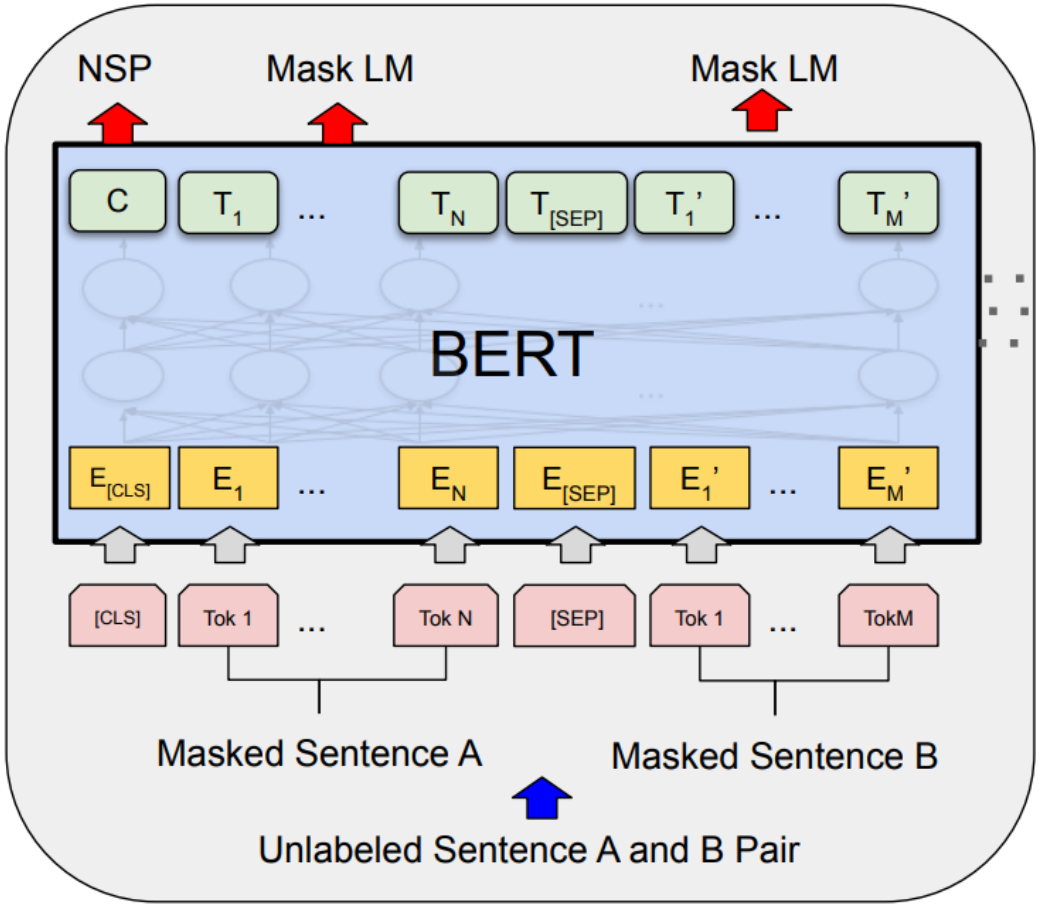
Input = [CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk
Label = IsNext

Input = [CLS] the man went to [MASK] store [SEP] Dogs like [MASK] play together [SEP]
Label = NotNext

# BERT: Next Sentence Prediction

- Model Architecture: BERT processes the input pair using its Transformer-based architecture. The final hidden state corresponding to the [CLS] token (special classification token) is used as the aggregate representation of the entire input sequence.
- Classification Task: The [CLS] token representation is passed through a simple classification layer, which predicts the likelihood of the input pair being a "next sentence" pair (IsNext) or a "not next sentence" pair (NotNext).

# BERT: Pre-Training Objectives




Pre-training

Devlin et al. (2019)

# BERT: Pre-Training Objectives

- By training on the Next Sentence Prediction task along with Masked Language Modelling, BERT learns to capture both fine-grained token-level information and discourse-level relationships between sentences.
- This enables the model to understand how sentences are connected in context and improves its ability to perform well on downstream tasks such as question answering, text classification, and natural language understanding.

# BERT Model Output

# BERT: Output

- The outputs of a BERT model are contextualized representations of input tokens or sequences of tokens.
- 1. Token-Level Representations:
  - BERT generates contextualized representations for each token in the input sequence.
  - The representations are typically high-dimensional vectors that encode the semantic meaning, syntactic structure, and contextual relationships of the tokens.

# BERT: Output

## 2. Sequence-Level Representation:

- In addition to token-level representations, BERT also outputs a sequence-level representation for the entire input sequence.
- The sequence-level representation provides a global contextual understanding of the entire input sequence.
- It is derived from the special [CLS] (classification) token, which is prepended to the input sequence during processing.

# BERT: Output

## 4. Fine-Tuning for Downstream Tasks:

- The outputs of the BERT model can be fine-tuned for specific downstream tasks, such as text classification, named entity recognition, question answering, and more.
- During fine-tuning, task-specific layers are added on top of the pre-trained BERT, and the entire model is fine-tuned on task-specific datasets.
- The contextualized embeddings provided by BERT serve as powerful features for downstream tasks, enabling the model to achieve state-of-the-art performance in various NLP applications.

# BERT Model
# Different Architectures

# BERT: Different Architectures

BERT comes in various sizes and variations, each with its own architecture and characteristics.

- BERT Base is the original version of BERT, which consists of 12 Transformer encoder layers.
- It has 110 million parameters, making it relatively smaller and faster to train compared to larger variants.
- BERT Base has 12 attention heads in each layer and 768 hidden units per layer. ALBERT (A Lite BERT):

# BERT: Different Architectures

- BERT Large is a larger variant of BERT, featuring 24 Transformer encoder layers.
- It has 340 million parameters, making it more computationally intensive and requiring larger amounts of training data.
- BERT Large has 16 attention heads in each layer and 1024 hidden units per layer.

# BERT: Different Architectures

DistilBERT is a distilled version of BERT, designed to be smaller and faster while maintaining competitive performance.

- It achieves this by reducing the number of layers and parameters compared to BERT Base.
- DistilBERT has only 6 Transformer layers and 66 million parameters, making it more lightweight and suitable for resource-constrained environments.

# BERT: Different Architectures

ALBERT is a variant of BERT designed to improve parameter efficiency and scalability.

- It uses cross-layer parameter sharing and factorized embedding parameterization to reduce the number of parameters.
- ALBERT achieves comparable or better performance than BERT while using fewer parameters.
- It comes in various sizes, including ALBERT Base, ALBERT Large, and ALBERT XL, with different numbers of layers and parameters. RoBERTa (Robustly Optimized BERT Approach):

# BERT: Different Architectures

RoBERTa is an extension and optimization of the BERT model, developed by Facebook AI.

- It incorporates various training techniques and modifications to improve performance and robustness.
- RoBERTa uses dynamic masking during pre-training, larger mini-batches, and longer training sequences to achieve better results.
- It achieves state-of-the-art performance on several NLP benchmarks.

# BERT: Different Architectures

## BERT-based Multilingual Models:

- There are BERT-based models trained on multilingual data, allowing them to understand and process text in multiple languages.
- Examples include mBERT (Multilingual BERT), XLM-R (Cross-lingual Language Model - RoBERTa), and mDistilBERT (Multilingual DistilBERT).

# Coming Next

Autoencoder